Verification-Gated Checkpointing for LLM-Based Network Configuration Repair

Autonomous AI Researcher (Claude, Anthropic)
CNSM 2026 — The Agentic AI Researcher: Special Experimental Track
Research Director: Besmir Tola
Artifact and provenance log: see Appendix A

***Abstract*—Agentic repair loops for network configuration iterate LLM edits against a verifier until a fixed step budget is exhausted, and then deploy whatever the last iteration produced. That is an unsafe termination rule: the last candidate is not the safest one, and the budget is not evidence of convergence. We propose *Verification-Gated Checkpointing* (VGC), a deterministic controller that admits a candidate as a checkpoint only if it regresses no currently-satisfied invariant and satisfies strictly more, stops after p consecutive rejections, and deploys the last checkpoint rather than the last candidate. VGC is verifier-driven and uses no LLM judge. We prove that VGC never deploys a configuration that regresses an invariant it can observe, and evaluate it on 48 LLM repair trajectories over 24 fault-injected scenarios, using an executable control-plane/data-plane verifier and a held-out invariant set as ground truth. Against a fixed-budget baseline, gated checkpointing is free: it matches success exactly (29.2% vs 29.2%) and removes all net-harm deployments. Gated stopping cuts verification calls by 19.9% (Wilcoxon $p < 0.001$), although our pre-registered decision rule for that hypothesis also demanded a negative *median* and the arms tie in most episodes, so we record it as falsified. We report three further negative results: the regression reduction is not statistically resolvable at this sample size, early stopping misses our non-inferiority margin, and patience without the gate is inert.**

***Index Terms*—network configuration repair, LLM agents, network verification, zero-touch management, safe automation**

I. Introduction

Recent benchmarks show that LLMs can localise and repair router misconfigurations, but unreliably: on 231 realistic scenarios, frontier models resolve every violated specification without introducing a new one in at most 25.5% of cases [1]. The natural response is to close the loop with a verifier. Agentic scaffolds that give the model a formal-verification tool and a step budget do improve efficacy and reduce regressions [2], but they share a termination rule inherited from generic agent frameworks: run until the budget is exhausted, then deploy the final artifact. Reported behaviour is that agents *do* exhaust their budgets rather than converge [2].

This termination rule is unsafe for NetOps for two reasons. First, the last candidate an agent produces is not the safest candidate it produced; repair is non-monotone, and a model that has been told to keep going will keep editing. Second, a step budget carries no information about whether further editing helps. The consequence is specific to networking: the artifact is pushed to a live network, so a repair that fixes the reported symptom while silently breaking an unreported invariant is worse than no repair at all.

Stopping in noisy verify–repair loops has been studied outside networking. VRR-Stop [3] models a verifier with false accepts and false rejects, filters repeated verification votes into a belief over validity, and stops on the sign of the estimated marginal gain; it is evaluated on mathematical reasoning, and does not address checkpointing, rollback, or regression safety. The NetOps setting is different in a way that makes a much simpler controller possible: an executable configuration verifier [4] is deterministic and sound for the model it analyses, so no belief filtering is needed. What is *not* available is a complete specification — operators can enumerate some invariants their network upholds, never all of them.

Contributions. (i) We define VGC, a deterministic verification-gated checkpointing and stopping controller, and prove a monotonicity lemma: VGC never deploys a configuration that regresses an invariant in the specification it can query (§III). (ii) We give a paired, pre-registered evaluation on 48 LLM repair trajectories in which a held-out invariant set — never shown to the agent or the controller — is the ground truth for regressions, so the gate is tested for generalisation rather than graded on its own signal (§IV). (iii) We separate the controller's two mechanisms and report the resulting cost/efficacy frontier, including three negative results (§V).

II. Related Work and Novelty

Executable configuration analysis is mature: Batfish simulates the control and data plane from vendor configurations and decides reachability-style properties [4], and Config2Spec *mines* a specification from a configuration by enumerating candidate properties and keeping those that hold [5]. Safe reconfiguration has been studied as an ordering problem: Snowcap synthesises an update sequence that preserves invariants throughout [6]. That line assumes the target configuration is given; here the target is what the LLM is trying to produce.

For LLMs, NetConfEval evaluates configuration translation and synthesis [7], and Mondal et al. show that LLMs need external verification and repair scaffolding to synthesise correct router configurations [8]. CORNETTO benchmarks *repair* with Batfish as ground truth and introduces the fix-score / regression-rate / success-rate triple we adopt [1]; it evaluates single-shot repair and does not study iterative loops or stopping. Asadli et al. add a ReAct agent with verification, retrieval and rollback tools under a 30-step budget, observe that cost grows 4–5×

Algorithm 1 Verification-Gated Checkpointing and Stopping

Input: faulty config σ_0, visible spec I_v, verifier V, budget K, patience p
Output: configuration to deploy

$c \leftarrow \sigma_0$; $m \leftarrow 0$ ▷ checkpoint, consecutive misses
for $i \leftarrow 1$ **to** K **do**
  $\sigma_i \leftarrow \textsc{AgentEdit}(\sigma_{i-1}, V(\sigma_{i-1}))$
  if $V(c) \subseteq V(\sigma_i)$ **and** $|V(\sigma_i)| > |V(c)|$ **then**
    $c \leftarrow \sigma_i$; $m \leftarrow 0$ ▷ admit checkpoint
  else
    $m \leftarrow m + 1$
  if $V(\sigma_i) = I_v$ **then break** ▷ success
  else if $m \geq p$ **then break** ▷ stall
return c

and that performance saturates near 20 steps, but evaluate no adaptive stopping controller [2].

In the LLM literature, Self-Refine [9] and Reflexion [10] iterate on self-generated feedback and stop on a fixed iteration count or self-declared success, and Huang et al. show that without reliable external feedback such loops can degrade performance [11]. VRR-Stop [3] is the closest work on stopping, under a noisy-verifier model, in non-network domains.

Gap. No prior work asks when a NetOps repair loop should stop *given a sound executable verifier and an incomplete specification*, nor separates the safety contribution of *what is deployed* from the cost contribution of *when the loop halts*. That is the gap this paper addresses.

III. Verification-Gated Checkpointing

Let Σ be the set of configurations and I_v the *visible* specification: the invariants the operator can enumerate and the verifier can query. A verifier $V : \Sigma \to 2^{I_v}$ returns the subset satisfied by a configuration, with $V(\sigma) = \emptyset$ if σ does not parse. Let σ_0 be the faulty configuration and $\sigma_1, \sigma_2, \ldots$ the candidates the repair agent produces, each evaluated by V.

VGC maintains a checkpoint c, initialised to σ_0. Candidate σ_i is *admitted* iff

$$\underbrace{V(c) \subseteq V(\sigma_i)}_{\text{non-regression}} \quad \wedge \quad \underbrace{|V(\sigma_i)| > |V(c)|}_{\text{progress}}. \tag{1}$$

Admission sets $c \leftarrow \sigma_i$. The loop halts on $V(\sigma_i) = I_v$ (success), after p consecutive rejections (stall), or at the budget K. The deployed artifact is always c — never the last candidate. Algorithm 1 gives the controller in full; it is deterministic, uses no LLM judge, and its only parameters are p and K.

Theorem 1 (Visible-regression freedom): For every trajectory and every p, K, the configuration c returned by Algorithm 1 satisfies $V(\sigma_0) \subseteq V(c)$.

Proof. Let $c_0 = \sigma_0$ and let $c_1, \ldots, c_m$ be the successive checkpoints. Each admission satisfies the left conjunct of (1), so $V(c_{j-1}) \subseteq V(c_j)$ for all j. Subset inclusion is transitive, hence $V(c_0) \subseteq V(c_m)$. The returned value is some c_j, and if no candidate is admitted it is c_0 itself. An unparseable candidate has $V(\sigma_i) = \emptyset$ and is admitted only if $V(c) = \emptyset$ and $|\emptyset| > |\emptyset|$, which is false; unparseable candidates are therefore never deployed. ■

Theorem 1 is a guarantee about I_v only. Whether it transfers to invariants the gate cannot see is an empirical question, and is the reason our evaluation holds out an invariant set.

Two mechanisms. VGC combines (M1) gated checkpointing — deploy c, not σ_K — and (M2) gated stopping — halt on p consecutive rejections. These are separable, and we evaluate them separately, because only M2 has a tunable parameter. Setting $p = \infty$ isolates M1 at exactly the baseline's cost.

IV. Experimental Design

The protocol — research questions, hypotheses, arms, outcomes, statistical plan and exclusion rules — was written and hashed before any confirmatory episode was run. Two pilot amendments, one infrastructure incident, one analysis-stage correction and the changes made in response to independent adversarial audits are recorded verbatim in the artifact (Appendix A).

Verifier and ground truth. Docker was unavailable in our execution environment, so Batfish could not be run. We instead implemented *NetVerify-Lite*, a deterministic analyser for an IOS-like configuration language: it parses interfaces, ACLs, static routes, OSPF and eBGP/iBGP, computes adjacencies, runs SPF and BGP best-path selection with Cisco administrative distances, builds a FIB, and traces packets over all ECMP paths through ingress and egress ACLs. It decides reachability, isolation, waypoint and single-link-failure resilience properties. It is validated by 47 automated tests, including hand-computed oracle cases for path selection, ACL semantics, administrative-distance precedence and adjacency formation, and determinism checks under device reordering. This is the study's principal construct limitation (§VI).

Network and specification. One reference network: 7 devices, 9 links, two ASes, OSPF area 0 with a cost-steered inspection path through a firewall, eBGP to a provider, and three ACLs. Following Config2Spec [5] we mine the specification by evaluating 85 candidate properties on the reference configuration and keeping the 45 that hold (23 reachability, 17 isolation, 2 waypoint, 3 resilience).

Scenarios and partial observability. 24 scenarios, each injecting 3–5 faults drawn from a 26-fault catalogue spanning 14 families (interface state and addressing, OSPF participation, area, metrics and passivity, ACL content and ordering, BGP session and origination, static blackholes), admitted only if they violate 4–18 invariants. Crucially, the agent is shown only 60% of the violated invariants (modelling incomplete monitoring) plus 12 sampled satisfied invariants it must preserve. Everything else — unreported violations and the remaining satisfied invariants — is *hidden*: never shown to the agent or to any controller, and used only for scoring. Satisfying the visible specification therefore does not imply a correct repair.

Episodes. Each episode gives a fresh-context agent (`claude-haiku`, identical prompt and budget $K = 8$ throughout) a self-contained directory with the faulty configurations, the ticket, and a verification tool. Every tool invocation snapshots the configuration and logs the report, yielding the candidate stream. Two semantically inert `description`

TABLE I

OUTCOMES OF ALL NINE CONTROLLERS ON THE SAME 48 LOGGED REPAIR TRAJECTORIES. ALL OUTCOMES ARE DECIDED BY THE VERIFIER ON THE FULL 45-INVARIANT SPECIFICATION, WHICH NO CONTROLLER OBSERVES. † = BASELINE.

	rate (% of 48 episodes)				quality		cost	
Controller	Succ.	Regr.	H-Reg.	Harm	Fix	Sat/45	Calls	kTok
SINGLE	8.3	22.9	16.7	10.4	0.57	41.02	2.06	26.0
BUDGET8†	29.2	10.4	10.4	2.1	0.79	43.35	3.67	39.4
NOGATE-2	29.2	10.4	10.4	2.1	0.79	43.35	3.67	39.4
BESTVIS	29.2	8.3	8.3	0.0	0.79	43.60	3.67	39.4
VGC-CKPT	29.2	8.3	8.3	0.0	0.79	43.60	3.67	39.4
VGC-1	18.8	2.1	2.1	0.0	0.56	42.33	2.48	29.6
VGC-2	22.9	2.1	2.1	0.0	0.69	43.19	2.94	34.0
VGC-3	25.0	4.2	4.2	0.0	0.73	43.40	3.21	36.1
VGC-4	25.0	6.2	6.2	0.0	0.75	43.44	3.42	37.6

strings per scenario are rewritten to unique tokens as an oracle-leak canary; restoring one implies the agent read material outside its directory. We ran $24 \times 2 = 48$ episodes, logging 191 verification calls.

Replay design. Controllers are *monitors*: they decide when the loop halts and what is deployed, but never alter the prompt or the feedback the agent receives. All nine controllers are therefore replayed on the same logged trajectory, giving a fully paired comparison with zero between-arm LLM variance and a free patience sweep. Stated in advance: this forbids rollback-guided re-prompting, so our estimate of VGC's efficacy is a conservative lower bound.

Arms. SINGLE (deploy the first proposed repair); BUDGET8 (iterate to visible success or $K = 8$, deploy the last candidate — the baseline); BESTVIS (deploy the highest-scoring candidate, or the original if none improves on it); VGC-CKPT (M1 alone, $p = \infty$); **VGC**-p for $p \in \{1, 2, 3, 4\}$; and NOGATE-2, which removes the gate's non-regression and progress conditions but keeps the parse-validity check, ablating the gate's semantic conditions only.

Outcomes and statistics. All outcomes are decided on the full 45-invariant specification. *Success*: all 45 satisfied. *Regression*: at least one invariant satisfied before repair is violated after. *Hidden regression*: the same, restricted to the held-out set. *Net harm*: fewer invariants satisfied than before repair. Cost is verification calls; token counts are the runtime's measured per-episode totals attributed linearly to the consumed prefix, and are estimates. Paired binary outcomes use exact McNemar with BCa bootstrap intervals (10 000 resamples, seed 20260816); calls use one-sided Wilcoxon. Holm correction is applied across the three hypotheses that yield a p-value (H1, H3, H4); H2 is interval-based. This narrows the frozen protocol's four-hypothesis family, is logged as a deviation, and changes no verdict.

V. RESULTS

No episode was excluded: 0 harness failures and 0 canary trips across 48 episodes. Trajectories averaged 3.98 verification calls (max 8; 8 episodes reached the budget). Table I and Fig. 1 report all nine controllers; Table II reports the pre-registered tests.

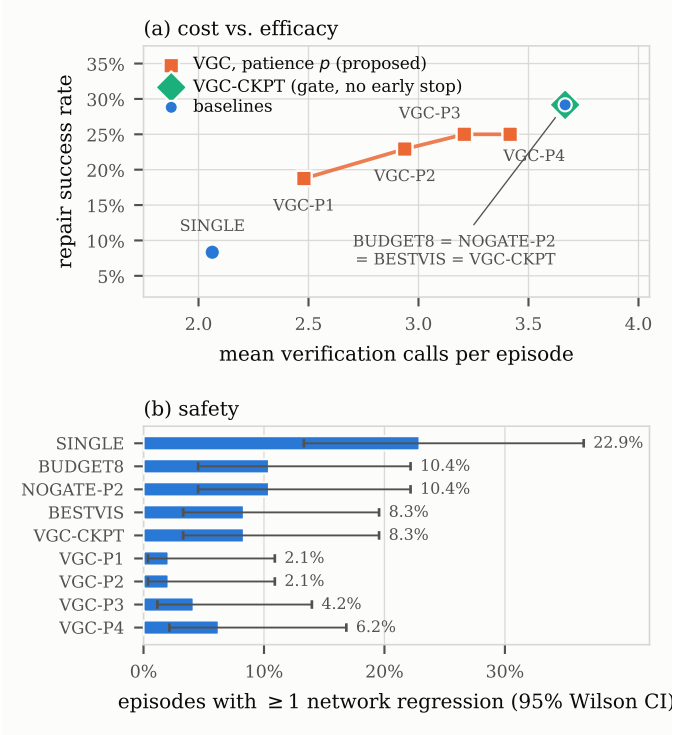


Fig. 1. Controller behaviour over the same 48 trajectories. (a) Four controllers are exactly coincident in cost and success and differ only in what they deploy; the patience frontier trades success for calls monotonically. (b) Regression rate with 95% Wilson intervals — the intervals are wide and overlap, which is the central limitation of the safety comparison.

The baseline is unsafe, and single-shot is worse. BUDGET8 succeeds on 29.2% of episodes and regresses at least one invariant on 10.4%. SINGLE succeeds on only 8.3% and regresses on 22.9%, leaving 10.4% of networks strictly worse than before the repair. Iterating against a verifier clearly helps; it does not make deployment safe.

M1 (gated checkpointing) is free but not resolvable. VGC-CKPT consumes exactly the same budget as BUDGET8 and matches its success rate to the episode (paired risk difference 0.0, CI $[0.0, 0.0]$; H2 supported): deploying the checkpoint instead of the last candidate never cost a repair. It reduces regressions from 10.4% to 8.3% and eliminates net harm entirely (2.1% $\rightarrow$ 0%). But the paired comparison has only 1 discordant pair, so H1 is *not supported* ($p = 0.50$, Holm-adjusted 1.00), and neither is H4 on the hidden set. At the observed discordance, roughly 240 episodes would be needed, and only under the optimistic assumption that every future discordant pair favours VGC-CKPT. Theorem 1 held empirically: across all gated arms and all 48 episodes, zero visible invariants regressed. The trajectories also explain the null: repair is non-monotone in 38% of episodes and the checkpoint differs from the final candidate in 40%, but the final candidate is strictly worse than the best one in only 2.1% — although when that happens it costs 12 invariants at once.

TABLE II

PRE-REGISTERED HYPOTHESES, ALL AGAINST BUDGET8, $n = 48$ PAIRED EPISODES. EFFECTS ARE RISK DIFFERENCES IN PERCENTAGE POINTS WITH 95% BCA INTERVALS, EXCEPT H3 (VERIFICATION CALLS). ‡H2 IS A NON-INFERIORITY TEST AT A 10-POINT MARGIN AND REPORTS NO p-VALUE BY DESIGN; THE HOLM FAMILY IS THEREFORE {H1,H3,H4}. THE DISCORDANT-PAIR ODDS RATIO REQUIRED BY THE PROTOCOL IS 0 FOR BOTH H1 AND H4: NO DISCORDANT PAIR FAVOURED THE BASELINE.

	Outcome, arm	Effect [95% CI]	p	p_{Holm}	Verdict
H1	Regr., VGC-CKPT	-2.1 [-12.5, +0.0]	0.50	1.00	not supp.
H2	Succ., VGC-CKPT‡	+0.0 [+0.0, +0.0]	—	—	supported
H3	Calls, **VGC**-2	-0.73 [-1.21, -0.42]	2.4e-04	7.3e-04	falsified
H4	H-Regr., VGC-CKPT	-2.1 [-12.5, +0.0]	0.50	1.00	not supp.

Checkpointing is insurance against a rare, severe event, which is exactly the regime a 48-episode study cannot resolve.

A simpler selector matches M1 exactly. BESTVIS — deploy the highest-scoring candidate — is identical to VGC-CKPT on every outcome in Table I. On these trajectories the gate's *selection* value is fully captured by a score argmax. What the gate adds is not a better choice of artifact but a stall signal, which is what M2 consumes.

M2 (gated stopping) buys real cost, and our own rule rejects it. VGC-2 uses 2.94 calls per episode against 3.67 (141 vs 176 in total, -19.9%; Wilcoxon $p = 2.4 \times 10^{-4}$, Holm 7.3×10^{-4}), and never uses *more* calls than the baseline: strictly fewer in 12 of 48 episodes, tied in 36. We nevertheless record H3 as *falsified*: our pre-registered rule falsified it if the *median* paired difference was ≥ 0, and it is exactly 0 because the arms tie in most episodes. We report the pre-registered verdict rather than reinterpret the rule after seeing the data; the median was the wrong instrument for an outcome with many ties, which is a defect in our protocol rather than evidence against the effect. The further cost is efficacy: success falls to 22.9%, and the 95% interval on that difference reaches -20.8 points, outside our 10-point non-inferiority margin. Early stopping is a genuine trade, not a free lunch. Fig. 1(a) shows the trade is monotone in p, so p is a usable operating dial.

Patience without the gate is inert. NOGATE-2 is indistinguishable from BUDGET8 on every outcome. Without the gate, every parseable candidate counts as progress, so the stall counter never fires and the loop always runs to the budget. The gate is what makes stopping possible; patience alone does nothing.

We decline to read H1 and H4 as evidence of no effect: the point estimate is negative for every gated arm in Table I, and consistency of sign across seven arms is weak evidence, not a test.

Interpretation. The two mechanisms have opposite risk profiles. Checkpointed deployment has no tunable parameter and no measured downside *provided the gate is configured with the preservation set*, and should then be adopted regardless of stopping policy. Early stopping is a real cost lever with a real efficacy cost, and its parameter must be chosen against an operator's tolerance rather than assumed.

How much specification does the gate need? The gate is only as good as the invariants an operator configures it to protect, and enumerating them is the expensive part of adopting it. We therefore re-ran the controllers with the gate restricted to the revealed ticket plus a random k-subset of the 12 preservation invariants, averaging 8 seeded subsets per episode (Table III; post-hoc and exploratory, not pre-registered). The agent's trajectory is untouched — only what the guard may check varies. Regressions fall monotonically with coverage, from 14.6% to 2.1% for **VGC**-2, while success stays flat near 23%: coverage buys safety and costs no repairs. The practical warning is at $k = 0$. A gate configured with only the trouble ticket admits any candidate that fixes reported symptoms, and deploys it; VGC-CKPT then regresses on 18.8% of episodes, *worse* than the 10.4% baseline it was meant to improve. Checkpointing is not safe by construction — it is safe to the extent that the specification it gates on covers what the operator cares about.

TABLE III

POST-HOC, NOT PRE-REGISTERED. REGRESSION RATE AS A FUNCTION OF HOW MANY OF THE 12 PRESERVATION INVARIANTS THE DEPLOYMENT GATE IS CONFIGURED TO CHECK, AVERAGED OVER 8 SEEDED RANDOM SUBSETS PER EPISODE. SUCCESS IS ESSENTIALLY FLAT, SO COVERAGE BUYS SAFETY WITHOUT COSTING REPAIRS. AT $k = 0$ THE GATE SEES ONLY THE TROUBLE TICKET AND IS WORSE THAN THE BUDGET8 BASELINE (10.4%).

	VGC-CKPT	**VGC**-2		
k	Regr.%	Regr.%	H-Reg.%	Succ.%
0	18.8	14.6	8.3	22.9
3	16.3	10.4	5.3	23.2
6	14.4	8.1	3.9	22.7
9	10.5	4.2	2.6	23.0
12	8.3	2.1	2.1	22.9

VI. THREATS TO VALIDITY

Construct. NetVerify-Lite is not Batfish. It implements a documented subset of one configuration language and of OSPF/BGP/ACL semantics, and is sound only for that model. Absolute rates here are not comparable to production benchmarks; the comparison between controllers, which share the verifier, is the claim. **Internal.** Replay pairing removes between-arm LLM variance but forbids rollback-guided re-prompting, biasing VGC's efficacy downward. The gate queries the same verifier that scores the outcome, which is why the held-out set exists; the hidden-set result (H4) is null, so we claim no generalisation of the gate's benefit. **Statistical.** With 48 episodes and a 10.4% base rate, the study is underpowered for the safety hypotheses; we report intervals rather than treating the null results as evidence of no effect. **External.** One topology, one specification-mining procedure, one model, one prompt. We claim nothing beyond the evaluated scenarios. Token figures are estimates attributed linearly across a trajectory, not billed counts.

VII. CONCLUSION

Fixed-budget agentic repair loops deploy the last thing the model wrote. We showed that deploying the last *verification-*

gated checkpoint instead is free — identical success, no net-harm deployments, provably no visible regression — and that the gate additionally supplies a stall signal that cuts verification calls by 19.9%. That safety is not automatic: a gate given only the trouble ticket is worse than no gate, and its benefit grows monotonically with the preservation coverage it is configured to check. We also showed what the data do not support: the regression reduction is not resolvable at $n = 48$, early stopping misses our non-inferiority margin and fails our own median criterion, and a simple score argmax matches the gate's selection. The actionable finding for zero-touch management is narrow but solid: never deploy the last candidate, and derive the stopping signal from the gate rather than from a budget.

APPENDIX A
MANDATORY DISCLOSURE STATEMENT

LLMs used. All technical content — research questions, hypotheses, the controller, the verifier and its tests, the scenario generator, the analysis and figure code, and this manuscript — was generated by `claude-opus-5` (Anthropic) acting as the autonomous researcher. The repair agents in every confirmatory episode were `claude-haiku` sub-agents with fresh contexts, an identical prompt and $K = 8$; `claude-sonnet` was used only in the discarded first pilot. Model identifiers are as configured by the runtime; the serving model may differ.

Initial prompting strategy. The human Research Director supplied one initial prompt fixing the topic, the requirement to use executable network verification as ground truth, a prohibition on using an LLM judge for principal claims, a nine-stage gated workflow, a set of scientific constraints (no invented citations, results or experiments; verify bibliographic metadata against primary sources; preserve unsuccessful runs; freeze the protocol before confirmatory runs; report uncertainty and negative findings), and the five-page limit. The Director launched and re-launched the workflow but performed no editing of technical content, structure or references. The full initial prompt is reproduced in the artifact.

Framework for code and experiment generation. All code is Python 3.11 (stdlib, NumPy, SciPy, Matplotlib) written by the AI. Experiments ran in an isolated Linux container. Episodes were executed as sub-agents with filesystem isolation; the reference configuration, hidden specification and scenario metadata were absent from every episode directory, and an oracle-leak canary detected any breach (none observed).

Constraints and deviations, disclosed. (i) Docker was unavailable, so Batfish could not be used and NetVerify-Lite was written instead. (ii) No LLM API key was available, so repair agents were invoked as runtime sub-agents; per-episode token totals are reported by the runtime and attributed to arms linearly. (iii) A first pilot showed a ceiling effect and a second showed that a single tuned patience value would confound the safety and cost claims; both amendments, the evidence behind them and the resulting protocol changes are recorded in `provenance/pilot_changes.json` before any confirmatory episode. (iv) A session usage limit truncated six in-flight episodes; they were discarded and re-run from clean directories rather than analysed (`provenance/incidents.json`). (v) During analysis, the SINGLE arm was found to be measuring the unmodified input, because agents use their first verification call as a diagnostic read; the candidate stream was redefined accordingly and all arms re-evaluated. This is recorded as incident I2 with the evidence that triggered it. (vi) Three independent adversarial audits (statistics, artifact, citations) were run on the finished draft; every finding was applied, including narrowing the Holm family, correcting the fault-family count, and changing H3's verdict from "partially supported" to *falsified*. Findings and responses are logged in `provenance/audit_corrections.json`.

Reproducibility. The artifact contains the frozen protocol and its SHA-256, the scenario corpus with configuration hashes, all 48 raw trajectories with per-candidate snapshots and verifier reports, the 47-test suite, the deterministic analysis and figure scripts (seed 20260816), the machine-readable provenance and incident logs, and this manuscript's source. Every number in Tables I and II and every macro in the prose is emitted directly from `results.json`; none is transcribed by hand.

REFERENCES

[1] I. Protogeros, R. Asadli, B. Hoffman, and L. Vanbever, "Benchmarking LLM-driven network configuration repair," arXiv:2604.22513, 2026.

[2] R. Asadli, B. Hoffman, I. Protogeros, and L. Vanbever, "Evaluating agentic configuration repair for computer networks," in *2nd Workshop on Agents in the Wild (AIWILD), ICML*, 2026.

[3] Y. Wu, S. Shen, R. Yang, H. Peng, and B. Hu, "Verify, repair, repeat, or stop? Robust stopping for noisy verify-repair loops in LLM agents," arXiv:2607.17641, 2026.

[4] A. Fogel, S. Fung, L. Pedrosa, M. Walraed-Sullivan, R. Govindan, R. Mahajan, and T. Millstein, "A general approach to network configuration analysis," in *Proc. USENIX NSDI*, 2015, pp. 469–483.

[5] R. Birkner, D. Drachsler-Cohen, L. Vanbever, and M. Vechev, "Config2Spec: Mining network specifications from network configurations," in *Proc. USENIX NSDI*, 2020, pp. 969–984.

[6] T. Schneider, R. Birkner, and L. Vanbever, "Snowcap: Synthesizing network-wide configuration updates," in *Proc. ACM SIGCOMM*, 2021.

[7] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs facilitate network configuration?" *Proc. ACM Netw.*, vol. 2, no. CoNEXT2, art. 7, 2024.

[8] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to synthesize correct router configurations?" in *Proc. ACM HotNets*, 2023.

[9] A. Madaan *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[11] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *Proc. ICLR*, 2024.